

Documentación Arquitectónica del Backend MVC (PHP)

1. Glosario Técnico y Conceptos Clave

PHP Data Objects (PDO)

PDO es la interfaz moderna y orientada a objetos de PHP para la interacción con bases de datos. Reemplaza funciones obsoletas (como `mysqli_query`) y provee una capa de abstracción robusta. Su mayor ventaja es la **mitigación nativa contra inyecciones SQL** mediante el uso de "Sentencias Preparadas" (*Prepared Statements*), operando como un mediador seguro entre la lógica de negocio y el motor de MySQL.

Data Source Name (DSN)

El DSN (`$dsn`) es una cadena de configuración que actúa como las coordenadas de conexión para PDO. Define el controlador del motor de base de datos (ej. `mysql`), el host, el nombre de la base de datos y el conjunto de caracteres (*charset*).

La Constante Mágica `__DIR__`

`__DIR__` es una constante predefinida en PHP que retorna la ruta absoluta del directorio donde reside el archivo actual en ejecución. Garantiza la portabilidad del código al resolver rutas relativas de manera dinámica, sin importar la configuración del servidor.

Gestión de Errores (Try-Catch) y PDOException

La estructura `try-catch` es el mecanismo estándar para el manejo de excepciones. Permite encapsular operaciones críticas (como conexiones a bases de datos) y capturar fallos operacionales. Si MySQL está inactivo o las credenciales son incorrectas, PDO emite una excepción `PDOException`, la cual es interceptada para evitar la exposición de trazas de error (stack traces) y presentar una respuesta controlada al usuario.

2. Modelos (Capa de Acceso a Datos)

Archivo: `modelos/Conexion.php`

Propósito: Proveer un enlace seguro y único hacia el motor de MySQL.

- **Patrón Singleton:** Implementado a través de una propiedad estática (`$instancia`) y un constructor privado. Garantiza que solo exista una única conexión a la base de datos activa durante el ciclo de vida de la petición HTTP, optimizando el consumo de recursos del servidor.
- **Configuración de Seguridad:** La conexión se envuelve en un bloque `try-catch` y se configuran atributos críticos de PDO, como el modo de reporte de errores (`PDO::ERRMODE_EXCEPTION`) y la desactivación de preparaciones emuladas, fortaleciendo la seguridad de las consultas.

Archivo: `modelos/Usuario.php`

Propósito: Manejar la persistencia y recuperación de datos de la entidad Usuario.

- **Inyección de Dependencias:** Al instanciarse, solicita inmediatamente el objeto PDO a la clase `Conexion`.
 - **Mitigación de SQL Injection:** El método `buscarPorEmail()` utiliza marcadores nominativos (`:email`) en lugar de variables directas. El flujo de ejecución es:
 1. `prepare()`: Envía la estructura de la consulta para pre-compilación.
 2. `execute()`: Asigna y sanea el valor real al marcador.
 3. `fetch(PDO::FETCH_ASSOC)`: Retorna el registro como un arreglo asociativo limpio.
-

3. Controladores (Capa de Lógica de Negocio)

Archivo: `publico/index.php` (Front Controller)

Propósito: Centralizar el punto de entrada de la aplicación, gestionando el enrutamiento y la carga de dependencias.

- **Seguridad Perimetral:** Evita el acceso directo a directorios internos del proyecto (ej. `/modelos` o `/vistas`).
- **Ruteo Dinámico (Routing):** Mediante parámetros HTTP GET (`c` para controlador y `a` para acción), procesa la URL y determina qué componente debe instanciarse.
- **Carga de Clases Dinámica:** Concatena el *namespace* base (`App\Controladores\`) con el nombre del controlador solicitado para realizar la instanciación al vuelo (`new $claseCompleta()`).

Archivo: `controladores/AuthControlador.php`

Propósito: Gestionar el flujo de autenticación, validación de credenciales y establecimiento de sesiones.

- **Control de Flujo HTTP:** Distingue el contexto de la petición evaluando `$_SERVER['REQUEST_METHOD']` (GET para presentar vistas, POST para procesar formularios).
- **Validación Criptográfica:** Emplea la función nativa `password_verify()` para comparar contraseñas en texto plano con hashes Bcrypt generados previamente (`$2y$10$...`), sin necesidad de descryptar el hash.
- **Manejo de Respuestas:** En caso de credenciales inválidas, inyecta la variable de error `$error` y rearma la vista, garantizando feedback visual al usuario.

Archivo: `controladores/AdminControlador.php`

Propósito: Proteger el área de administración y orquestar los datos proyectados en el tablero de control.

- **Seguridad en Constructor:** Al invocar `Sesion::requerirLogin()` dentro del constructor, se asegura de que ninguna acción de esta clase pueda ejecutarse si el usuario no posee una sesión activa.
-

4. Ayudantes y Servicios

Archivo: `ayudantes/Sesion.php`

Propósito: Orquestar el estado HTTP (*State Management*) y los roles del usuario.

- **Protección OWASP:**

- `session_regenerate_id(true)`: Mitiga ataques de *Session Fixation* mediante la rotación del ID de sesión tras el login.
 - Gestión de caducidad (*Timeboxing*) calculada mediante la función `time()`.
 - En la fase de `logout()`, además de destruir el arreglo `$_SESSION`, borra deliberadamente la cookie de sesión en el cliente, logrando un cierre completo y seguro.
 - **Almacenamiento de Metadatos:** Durante `iniciarLogin()`, persiste claves vitales (`usuario_id`, `rol_id`, `usuario_nombre`) para eludir viajes redundantes a la base de datos durante la navegación.
 - **Invalidación de Sesión:** Cuando ocurren modificaciones estructurales en la lógica de sesiones, es imperativo invalidar (logout) los tokens vigentes para obligar al sistema a rehidratar las variables de memoria.
-

5. Vistas (Capa de Presentación)

Adaptación de Vistas y Rutas Absolutas (`URL_BASE`)

La transición a un entorno MVC y *Front Controller* requiere que todas las referencias estáticas (CSS, JS, imágenes) abandonen los directorios relativos (`../`).

- **Estandarización de Rutas:** Se implementa la constante `URL_BASE` para anclar la búsqueda de *assets* desde la raíz del proyecto público.
- **Separación de Responsabilidades:** Se proscribe el uso de estilos en línea en los archivos PHP. Todo encapsulamiento estético reside en las hojas de estilo correspondientes, dejando los archivos de vista enfocados puramente en maquetación semántica e inyección de variables.